



**GAZİ ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**BMT210
VERİ YAPILARI
DERSİ DÖNEM PROJESİ
RAPORU**

Hastane Randevu ve Hasta Öncelik Yönetim Sistemi

Muhammet Eren Demirtaş-23181616052

Onur Olmuş-23181616050

Hayri Topaloğlu - 25181616752

DOÇ. DR. Adem TEKEREK

İÇİNDEKİLER

1. PROBLEM TANIMI	1
2. AMAÇ VE KAPSAM	2
• 2.1. Projenin Amacı	2
• 2.2. Kapsam	2
3. KULLANILAN VERİ YAPILARI	3
• 3.1. Hash Table (Hasta Kayıt Yönetimi)	3
• 3.2. Min-Heap (Acil Triage Yönetimi)	4
• 3.3. Queue (Normal Randevu Yönetimi)	5
• 3.4. Stack (İşlem Geçmişi ve Geri Alma)	6
• 3.5. N-ary Tree (Poliklinik Hiyerarşisi)	7
4. SİSTEM TASARIMI VE ALGORİTMALAR	8
• 4.1. Genel Mimari (C ve Web Arayüzü)	8
• 4.2. Temel Algoritmaların İşleyişi	9
5. TEST SENARYOLARI VE PERFORMANS	11
• 5.1. İşlevsel Test Tabloları	11
• 5.2. Performans Karşılaştırma Grafikleri	12
6. SONUÇ VE DEĞERLENDİRME	14
7. KAYNAKÇA	15

1. Problem Tanımı

Hastaneler, günlük operasyonlarında binlerce hasta kaydını yönetmek, randevu taleplerini sıralamak, acil vakaları önceliklendirmek ve tüm bu süreçleri anlık olarak izlemek zorundadır. Bu süreçlerin manuel ya da verimsiz veri yapılarıyla yönetilmesi; uzayan bekleme süreleri, yanlış önceliklendirme hataları ve idari iş yükü artışı gibi ciddi sonuçlar doğurabilmektedir. Özellikle acil servis ortamlarında, yanlış önceliklendirmenin doğrudan insan hayatını etkileyebileceği düşünüldüğünde, bu sistemlerin yalnızca işlevsel değil aynı zamanda algoritmik olarak doğru ve performanslı olması zorunluluk hâline gelmektedir.

Bu projede ele alınan temel problem; bir hastanenin hasta kayıt, randevu yönetimi ve acil triaj süreçlerini dijital ortamda modellemek ve bu modeli, farklı işlemlerin farklı performans gereksinimlerini karşılayacak biçimde uygun veri yapılarıyla desteklemektir.

Sistemin çözmesi beklenen başlıca problemler şu şekilde özetlenebilir:

Hızlı hasta erişimi: Binlerce kayıt arasında bir hastanın TC kimlik numarasıyla anlık olarak bulunabilmesi gerekmektedir. Doğrusal arama yöntemleri bu ölçekte kabul edilemez gecikmelere yol açar.

Adil randevu sıralaması: Normal randevu alan hastalar, başvuru sıralarına göre çağrılmalıdır. Sıranın bozulması hasta memnuniyetsizliğine ve idari şikâyetlere neden olur.

Acil önceliklendirme: Acil servise başvuran her hasta aynı aciliyette değildir. Triaj sistemi, hastanın klinik durumunu yansıtan bir skora göre her an en kritik hastayı öne alabilmelidir.

İşlem geri alınabilirliği: Yanlış yapılan kayıt ekleme, silme veya güncelleme işlemlerinin geri alınabilmesi, sistemin güvenilirliği açısından zorunludur.

Kurumsal yapı yönetimi: Hastane bünyesindeki poliklinikler ve alt birimler hiyerarşik bir yapıda modellenmelidir; bu yapıya birim eklenmesi veya çıkarılması mümkün olmalıdır.

2. Amaç ve Kapsam

2.1 Projenin Amacı

Bu projenin temel amacı, yukarıda tanımlanan hastane yönetim problemini yalnızca çalışan bir yazılım olarak değil, veri yapılarının bilinçli ve gerekçeli biçimde seçildiği bir mühendislik çalışması olarak ortaya koymaktır.

Bu doğrultuda proje iki katmanlı bir hedef taşımaktadır. İlk katmanda, gerçek bir hastane senaryosunu modelleyen, HTTP tabanlı bir backend ve web tabanlı bir admin panelinden oluşan tam işlevli bir yazılım sistemi geliştirilmiştir. İkinci katmanda ise her bir modülde tercih edilen veri yapısının neden seçildiği, alternatiflerine kıyasla hangi performans avantajlarını sağladığı ve ölçülebilir testlerle bu avantajların nasıl doğrulandığı ortaya konmuştur.

Projenin somut hedefleri şunlardır:

- Hasta kayıtlarının TC kimlik numarası üzerinden $O(1)$ ortalama karmaşıklıkla yönetilmesi
- Normal randevu akışının FIFO ilkesiyle, acil triajın ise öncelik skoruna dayalı olarak işletilmesi
- Yapılan her idari işlemin geri alınabilir biçimde kaydedilmesi
- Poliklinik hiyerarşisinin dinamik olarak değiştirilebilir ağaç yapısıyla modellenmesi
- Farklı veri yapılarının aynı problem üzerindeki performans farklarının ölçülerek raporlanması

2.2 Kapsam

Sistem; C programlama diliyle geliştirilmiş bir POSIX socket tabanlı HTTP sunucu ve HTML/CSS/JavaScript ile hazırlanmış bir web arayüzünden oluşmaktadır. Herhangi bir harici kütüphane ya da veritabanı motoru kullanılmamış; tüm veri yapıları ve sunucu altyapısı sıfırdan implement edilmiştir. Bu tercih, veri yapılarının iç işleyişinin tam olarak denetlenebilmesi ve performans ölçümlerinin dış bağımlılıklardan arındırılmış biçimde yapılabilmesi amacıyla alınmıştır.

Projenin kapsamı beş temel modül üzerine inşa edilmiştir:

Hasta Yönetimi Modülü, tüm hasta kayıtlarını Chaining yöntemiyle çakışma çözen bir Hash Table yapısında saklar. Polynomial rolling hash fonksiyonu kullanılarak 1009 asal boyutlu bir bucket dizisinde veriler dağıtılır. Bu yapı sayesinde ekleme, silme, arama ve güncelleme işlemlerinin tamamı $O(1)$ ortalama karmaşıklıkla gerçekleştirilmektedir.

Acil Triaj Modülü, gelen hastaları 1 ile 10 arasında bir aciliyet skoruyla dizi tabanlı Min-Heap yapısına ekler. Kök düğüm her zaman en düşük — yani en kritik — skora sahip hastayı tutar. En acil hastanın çağırılması $O(\log n)$ karmaşıklığıyla gerçekleşir.

Randevu Kuyruğu Modülü, normal başvuruları bağlı liste tabanlı bir Queue yapısıyla FIFO ilkesine göre yönetir. Hem enqueue hem dequeue işlemleri $O(1)$ karmaşıklığındadır.

İşlem Geçmişi Modülü, sistemde gerçekleştirilen her işlemi bağlı liste tabanlı bir Stack yapısına kaydeder. "Geri Al" işlevi, stack'in tepesindeki kaydı $O(1)$ karmaşıklığında alarak ilgili işlemi tersine çevirir.

Poliklinik Yönetimi Modülü, hastanenin kurumsal birim yapısını ebeveyn pointer içeren N-ary Tree yapısıyla modellemektedir. Ağaç üzerinde DFS gezinti, alt birim ekleme ve silme işlemleri desteklenmektedir.

Projenin kapsamı dışında tutulan konular şunlardır: kullanıcı kimlik doğrulama ve yetkilendirme, kalıcı veritabanı depolama, çoklu hastane desteği ve mobil uyumluluk. Sistem, tek bir hastane biriminin yönetimini hedefleyen, eğitim odaklı bir prototip niteliğindedir.

3. Kullanılan Veri Yapıları

Bu projede beş farklı veri yapısı kullanılmıştır. Her yapı, ilgili modülün işlevsel ve performans gereksinimlerine göre seçilmiş; alternatif yaklaşımlarla karşılaştırılarak gerekçelendirilmiştir.

Veri Yapıları: Seçim Gerekçeleri & Big-O

Yapı	Kullanım	Arama	Ekleme
Hash Table	Hasta kaydı erişimi	$O(1)$ ort.	$O(1)$ ort.
Min-Heap	Öncelik kuyruğu	$O(\log n)$	$O(\log n)$
N-ary Tree	Poliklinik hiyerarşisi	$O(n)$	$O(1)$
Queue	FIFO randevu sırası	—	$O(1)$
Stack	İşlem geçmişi / geri al	—	$O(1)$

3.1 Hash Table — Hasta Kayıt Yönetim+

3.1.1 Yapının Tanımı ve Çalışma Prensipleri

Hash Table, anahtarları bir hash fonksiyonu aracılığıyla dizi indekslerine dönüştürerek veriyi saklayan ve erişen bir veri yapısıdır. Bu projede hasta kayıtları, TC kimlik numarası anahtar olarak kullanılarak hash table'da saklanmaktadır.

Hash fonksiyonu olarak polynomial rolling hash tercih edilmiştir:

$$h(tc) = (\sum tc[i] \times 31^i) \bmod 1009$$

Burada 1009 asal bir sayı olarak seçilmiştir. Asal sayı kullanımı, anahtarların bucket'lara daha homojen dağılmasını sağlar; bu da çakışma sayısını minimize ederek ortalama zincir uzunluğunu düşürür.

Çakışma çözümü için **Chaining** yöntemi kullanılmıştır. Aynı bucket'a düşen kayıtlar, o bucket'a bağlı bir linked list zincirinde tutulur. Yük faktörü (load factor) $\text{toplam_hasta} / 1009$ formülüyle hesaplanmakta ve dashboard üzerinden canlı olarak izlenebilmektedir.

3.1.2 Neden Hash Table Seçildi — Alternatif Karşılaştırması

Hasta kaydı modülünün en kritik işlemi aramadır. Bir doktorun ya da hemşirenin, binlerce kayıt arasından belirli bir hastayı TC numarasıyla anında bulabilmesi gerekmektedir. Bu gereksinim, veri yapısı seçimini doğrudan belirlemiştir.

Veri Yapısı	Arama	Ekleme	Silme	Bellek
Hash Table	$O(1)$ ort.	$O(1)$ ort.	$O(1)$ ort.	Orta
Sıralı Dizi	$O(\log n)$	$O(n)$	$O(n)$	Düşük
Bağlı Liste	$O(n)$	$O(1)$	$O(n)$	Düşük
BST (dengeli)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Orta

Bağlı liste ile yapılan aramada, $n = 10.000$ kayıt için ortalama 5.000 düğüm gezilmesi gerekmektedir. Hash table'da ise aynı arama yük faktörü düşük tutulduğu sürece sabit sürede tamamlanır. Performans testleri bu farkı sayısal olarak doğrulamıştır; 10.000 kayıtlık veri setinde hash table ile yapılan arama, bağlı listeye kıyasla yaklaşık 2.115 kat daha hızlı sonuç vermiştir.

BST tercih edilmemesinin temel nedeni, hasta kayıtlarının alfabetik ya da sıralı erişim gerektirmemesidir. TC numarası ile doğrudan erişim söz konusu olduğunda BST'nin $O(\log n)$ karmaşıklığı, hash table'ın $O(1)$ ortalamasının gerisinde kalır. Ayrıca BST'nin dengeli kalması için AVL ya da Red-Black gibi ek düzenleme mekanizmaları gerektirir; bu da implementasyon karmaşıklığını artırır.

3.2.3 Zaman ve Yer Karmaşıklığı

İşlem	Karmaşıklık
En kritik hastayı görüntüle	$O(1)$
Hasta ekle (sift-up)	$O(\log n)$
En kritik hastayı çıkar (sift-down)	$O(\log n)$
Tüm listeyi gez	$O(n)$

3.3 Queue — Normal Randevu Yönetimi

3.3.1 Yapının Tanımı ve Çalışma Prensipleri

Queue, FIFO (First In First Out) ilkesiyle çalışan ve iki uçtan işlem yapılan doğrusal bir veri yapısıdır. Bu projede bağlı liste tabanlı bir queue implementasyonu tercih edilmiştir. `front` pointer'ı dequeue tarafını, `rear` pointer'ı ise enqueue tarafını göstermektedir. Bu çift pointer yapısı sayesinde her iki uç işlemi de $O(1)$ karmaşıklığında gerçekleştirilir.

3.3.2 Neden Queue Seçildi — Alternatif Karşılaştırması

Normal randevu sisteminin ihtiyacı nettir: ilk başvuran hasta ilk çağrılmalıdır. Bu FIFO semantiği, queue yapısını bu modül için biçilmiş kaftan hâline getirir.

Veri Yapısı	Enqueue	Dequeue	FIFO Garantisi
Queue (Linked List)	$O(1)$	$O(1)$	Evet
Dizi (sabit boyutlu)	$O(1)$	$O(n)$ — kaydırma	Evet
Dairesel Dizi	$O(1)$	$O(1)$	Evet
Yığın (Stack)	$O(1)$	$O(n)$ — ters çevirme	Hayır

Sabit boyutlu dizi kullanılsaydı, dequeue işleminde tüm elemanların bir sola kaydırılması gerekirdi; bu durum $O(n)$ karmaşıklığına ve yüksek hasta trafiğinde ciddi performans sorunlarına yol açardı. Dairesel dizi de $O(1)$ dequeue sağlayabilir; ancak sabit boyutlu olduğundan kapasitesi önceden belirlenmek zorundadır. Bağlı liste tabanlı queue ise dinamik büyüyebilir ve her iki uç işlemi $O(1)$ karmaşıklığında kalır.

Randevu iptali için TC numarasına göre $O(n)$ aramanın ardından $O(1)$ zincir güncellemesi yapılmaktadır. Bu işlem nadir gerçekleştiğinden $O(n)$ maliyeti kabul edilebilir düzeydedir.

3.3.3 Zaman ve Yer Karmaşıklığı

İşlem	Karmaşıklık
Randevu al (enqueue)	$O(1)$
Sıradaki hastayı çağır (dequeue)	$O(1)$
Randevu iptal (TC ile)	$O(n)$
Kuyruğu görüntüle	$O(n)$

3.4 Stack — İşlem Geçmişi ve Geri Alma

3.4.1 Yapının Tanımı ve Çalışma Prensipleri

Stack, LIFO (Last In First Out) ilkesiyle çalışan ve tek uçtan işlem yapılan bir veri yapısıdır. Bu projede bağlı liste tabanlı olarak implement edilmiştir. Sistemde gerçekleştirilen her işlem — hasta ekleme, silme, güncelleme, randevu alma ve poliklinik ekleme — işlem tipi, ilgili TC

numarası, açıklama ve zaman damgasıyla birlikte stack'e push edilir. "Geri Al" işlevi tetiklendiğinde stack'in tepesindeki kayıt pop edilerek ilgili işlem tersine çevrilir.

3.4.2 Neden Stack Seçildi — Alternatif Karşılaştırması

Geri alma mekanizmasının doğası gereği en son yapılan işlemin ilk geri alınması gerekir. Bu LIFO semantiği, stack'i bu modül için tek doğal seçim hâline getirir.

Veri Yapısı	Son işlemi bul	Push	Pop
Stack (Linked List)	$O(1)$	$O(1)$	$O(1)$
Dizi	$O(1)$	$O(1)$ / $O(n)$ yeniden boyutlama	$O(1)$
Queue	$O(n)$ — başa kadar gez	$O(1)$	$O(1)$ — ama yanlış uç
Bağlı Liste (sirasız)	$O(n)$	$O(1)$	$O(n)$

Queue ile geri alma yapılmak istenseydi, en son eklenen elemana ulaşmak için tüm kuyruğun geçilmesi gerekirdi. Bağlı listede ise "son eklenen" bilgisi ayrıca takip edilmediği sürece $O(n)$ arama kaçınılmazdır. Stack, tepe pointer'ı sayesinde bu erişimi her zaman $O(1)$ 'de sağlar.

3.4.3 Zaman ve Yer Karmaşıklığı

İşlem	Karmaşıklık
İşlem kaydet (push)	$O(1)$
Son işlemi geri al (pop)	$O(1)$
Son n işlemi görüntüle	$O(n)$

3.5 N-ary Tree — Poliklinik Hiyerarşisi

3.5.1 Yapının Tanımı ve Çalışma Prensipleri

N-ary Tree, her düğümün birden fazla çocuk düğüme sahip olabildiği hiyerarşik bir veri yapısıdır. Bu projede hastanenin kurumsal birim yapısı bu yapıyla modellenmiştir. Her düğüm; birim adını, en fazla MAX_COCUK sayıda çocuk pointer'ını ve ebeveynine doğrudan erişim sağlayan bir ebeveyn pointer'ını barındırmaktadır.

Ebeveyn pointer'ı, bir birimin silinmesi sırasında üst birime $O(1)$ karmaşıklığında erişimi mümkün kılar. Ağaç üzerinde DFS (Depth-First Search) gezintisi hem JSON serileştirmede hem de isme göre düğüm aramada kullanılmaktadır.

3.5.2 Neden N-ary Tree Seçildi — Alternatif Karşılaştırması

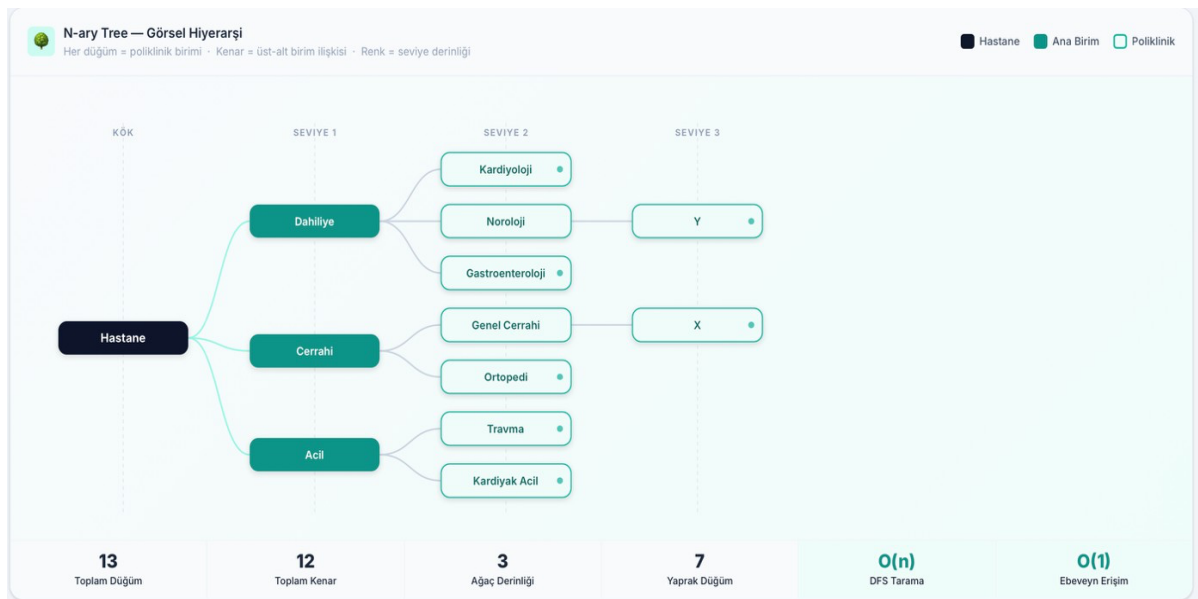
Poliklinik yapısının doğası hiyerarşiktir: bir ana birim birden fazla alt birim içerebilir ve bu ilişki birden fazla seviyeye yayılabilir. Bu ilişkiyi düz bir yapıyla modellemek hem anlamsız hem de verimsiz olurdu.

Yaklaşım	Alt birimleri bul	Ekleme	Silme	Hiyerarşi
N-ary Tree	$O(k)$ — doğrudan	$O(n)$ bul + $O(1)$ ekle	$O(n)$ bul + $O(k)$ sil	Doğal
Düz Liste (parent_id)	$O(n)$ — tüm listeyi tara	$O(1)$	$O(n)$	Dolaylı
Hash Map	$O(n)$ — ebeveyn kaybolur	$O(1)$	$O(n)$ ayrı tarama	Yok
Graf	$O(V+E)$	$O(1)$	$O(V+E)$ döngü kontrol	Aşırı karmaşık

Düz liste yaklaşımında belirli bir birimin alt birimlerini bulmak için tüm listenin taranması gerekir. Hash map ise ebeveyn bilgisini doğrudan saklayamaz ve silme işleminde ayrı bir tarama zorunlu hâle gelir. Graf yapısı ise döngü kontrolü gibi ek mekanizmalar gerektirdiğinden ağaç topolojisi için gereksiz karmaşıklık yaratır. N-ary Tree bu senaryoda hem semantik hem de performans açısından en uygun yapıdır.

3.5.3 Zaman ve Yer Karmaşıklığı

İşlem	Karmaşıklık
DFS ile düğüm arama	$O(n)$
Alt birimleri listele	$O(k)$ — k: çocuk sayısı
Birim ekle	$O(n)$ bul + $O(1)$ ekle
Birim sil (alt ağaçla)	$O(n)$ bul + $O(k)$ sil
JSON serileştirme	$O(n)$



4. Sistem Tasarımı

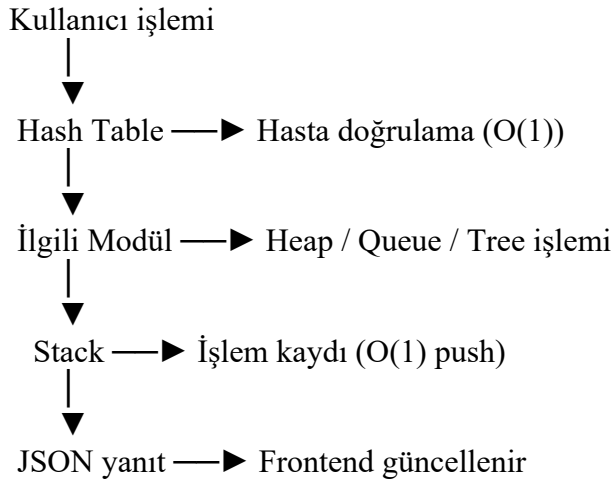
4.1 Genel M+mar+

Sistem, birbirinden bağımsız iki katmandan oluşmaktadır: C ile geliştirilmiş bir backend sunucu ve HTML/CSS/JavaScript ile hazırlanmış bir web arayüzü. Bu iki katman, JSON formatında HTTP mesajlaşmasıyla haberleşmektedir.

Backend, herhangi bir harici kütüphane veya veritabanı motoru kullanmaksızın POSIX socket API üzerine sıfırdan inşa edilmiştir. Her gelen bağlantı için pthread_create ile ayrı bir iş parçacığı oluşturulmakta; tek bir global pthread_mutex_t ile tüm veri yapılarına eş zamanlı erişim güvenli hâle getirilmektedir. JSON yanıtları harici bir kütüphane kullanılmadan tamamen manuel olarak oluşturulmaktadır.

4.2 Modüller Arası İl+şk+

Sistem modülleri birbirinden bağımsız çalışabilmekle birlikte, işlem akışı sırasında birbirlerini tetiklemektedir. Örneğin bir hasta triaj kuyruğuna eklendiğinde önce hash table'dan hasta bilgisi doğrulanmakta, ardından heap'e ekleme yapılmakta ve son olarak bu işlem stack'e kaydedilmektedir. Bu sayede geri alma mekanizması tüm modülleri kapsayacak biçimde çalışır.



4.3 Dosya Yapısı

4.4 API Tasarımı

```
Hastane
├── backend
│   ├── hash_table.c
│   ├── hash_table.h
│   ├── hash_table.o
│   ├── hastane_server
│   │   ├── heap.c
│   │   ├── heap.h
│   │   ├── heap.o
│   │   ├── main.c
│   │   ├── main.o
│   │   ├── Makefile
│   │   ├── queue.c
│   │   ├── queue.h
│   │   ├── queue.o
│   │   ├── stack.c
│   │   ├── stack.h
│   │   ├── stack.o
│   │   ├── tree.c
│   │   ├── tree.h
│   │   └── tree.o
│   └── data
│       ├── hastalar.txt
│       ├── randevular.txt
│       └── frontend
│           ├── app.js
│           ├── index.html
│           └── style.css
└── ...

## API Endpoint'leri
## Genel
GET /api/dashboard - Canlı istatistikler + kuyruk listesi (JSON)
GET /api/istatistikler - Hash Table yük analizi, poliklinik dağılımı, triaj skor histogramı, heap/stack dolulukları

## Hasta Yönetimi (Hash Table)
POST /api/hasta - Hasta ekle O(1) ort.
GET /api/hasta?tc=... - TC ile hasta ara O(1) ort.
GET /api/hastalar - Tüm hastaları listele O(n)
PUT /api/hasta/itc - Hasta güncelle O(1) ort.
DELETE /api/hasta/itc - Hasta sil O(1) ort.

## Randevu Kuyruğu (Queue)
POST /api/randevu - Kuyruğa ekle (enqueue) O(1)
GET /api/randevu/kuyruk - Kuyruğu görüntüle O(n)
POST /api/randevu/cagir - Sıradakini çağır (dequeue) O(1)
DELETE /api/randevu/itc - Randevu iptal O(n)

## Acil Triaj (Min-Heap)
POST /api/triaj - Triaja ekle (skor: 1-10) O(log n)
GET /api/triaj/kuyruk - Triaj listesi O(n)
POST /api/triaj/cagir - En acili çağır (pop) O(log n)

## Poliklinik Yönetimi (N-ary Tree)
GET /api/poliklinikler - Ağac yapısı (JSON) O(n)
POST /api/poliklinik - Yeni birim ekle O(n bul) + O(1 ekle)
DELETE /api/poliklinik/iad - Birim + alt ağacı sil O(n bul) + O(k sil)

## İşlem Geçmişi [Stack]
GET /api/gecmis - Son 50 işlem O(50)
POST /api/gecmis/geri-al - Son işlemi geri al O(1) pop + O(1) undo
```

4.5 Ver+ Akış Senaryoları

Senaryo 1 — Acil Hasta Kabulü: Acil servise gelen bir hasta sisteme kaydedilir (hash table'a $O(1)$ ekleme), ardından triaj skoruyla birlikte Min-Heap'e eklenir ($O(\log n)$ sift-up). İşlem stack'e kaydedilir. Dashboard'daki triaj listesi bir sonraki polling döngüsünde güncellenerek en kritik hasta kuyruğun başında görünür hâle gelir.

Senaryo 2 — Yanlış İşlemi Geri Alma: Admin panelinden yanlışlıkla silinen bir hasta kaydı, "Geri Al" butonuna basılmasıyla stack'ten pop edilir. Pop edilen kayıttaki undo verisi kullanılarak hasta hash table'a yeniden eklenir. Tüm süreç $O(1)$ karmaşıklığında tamamlanır.

Senaryo 3 — Normal Randevu Akışı: Randevu alan hasta queue'ya enqueue edilir ($O(1)$). Doktor muayene odasına geçmeye hazır olduğunda "Sıradaki Hastayı Çağır" butonuna basılır, queue'nun ön'ünden dequeue yapılır ($O(1)$) ve hasta bilgisi ekranda gösterilir.

5. Algoritmik Yaklaşım

Bu bölümde sistemdeki beş modülün temel algoritmalarının adım adım işleyişi açıklanmaktadır. Amaç, kodun arka planındaki algoritmik kararları ve bu kararların veri yapısı seçimleriyle ilişkisini ortaya koymaktır.

5.1 Hash Fonksiyonu ve Chaining

Hasta ekleme işleminde sırasıyla şu adımlar izlenir:

1. tc dizisi üzerinde polynomial rolling hash hesapla
 $h = 0$
her $tc[i]$ için: $h = (h * 31 + tc[i]) \bmod 1009$
 2. h indeksli bucket'a git
 3. Bucket boşsa → yeni düğüm oluştur, bucket'a bağla
Bucket doluysa → zincirin sonuna ekle (chaining)
 4. İşlemi stack'e kaydet
- Arama işlemi de aynı hash hesabıyla başlar; fark olarak zincir boyunca TC karşılaştırması yapılır. Silmede ise bulunan düğümün önceki düğümünün next pointer'ı güncellenerek zincirden koparılır.

5.2 Min-Heap — Sift-Up ve Sift-Down

Ekleme (Sift-Up):

1. Yeni hastayı dizinin sonuna ekle, boyutu artır
2. $i = \text{son indeks}$
3. $\text{Ebeveyn}(i) = (i-1)/2$
4. $\text{heap}[i].\text{skor} < \text{heap}[\text{ebeveyn}].\text{skor}$ iken:
 $\text{heap}[i] \leftrightarrow \text{heap}[\text{ebeveyn}]$ yer değiştir
 $i = \text{ebeveyn}$
5. Heap özelliği sağlandığında dur

En Acil Hastayı Çıkarma (Sift-Down):

1. Kök (en kritik hasta) kaydet $\rightarrow$ döndürülecek
2. Dizinin son elemanını köke taşı, boyutu azalt
3. $i = 0$
4. Sol ve sağ çocuktan küçük olanı bul $\rightarrow$ min_cocuk
5. $\text{heap}[i].\text{skor} > \text{heap}[\text{min_cocuk}].\text{skor}$ iken:
 $\text{heap}[i] \leftrightarrow \text{heap}[\text{min_cocuk}]$ yer değiştir
 $i = \text{min_cocuk}$
6. Heap özelliği sağlandığında dur
Her iki algoritma da $O(\log n)$ karmaşıklığında çalışır; çünkü tam ikili ağaçta derinlik her zaman $\lfloor \log_2 n \rfloor$ ile sınırlıdır.

5.3 Queue — Enqueue ve Dequeue

Enqueue:

1. Yeni düğüm oluştur
2. kuyruk boşsa $\rightarrow$ on = arka = yeni düğüm
 doluyrsa $\rightarrow$ arka- $\rightarrow$ next = yeni düğüm; arka = yeni düğüm

Dequeue:

1. on == NULL ise kuyruk boş $\rightarrow$ hata döndür
2. döndürülecek = on
3. on = on- $\rightarrow$ next
4. on == NULL ise arka = NULL (kuyruk boşaldı)
5. düğümü serbest bırak (free)
İptal işleminde TC eşleşmesi bulunana kadar zincir gezilir, bulunan düğümün önceki düğümünün next pointer'ı güncellenerek kuyruktan çıkarılır.

5.4 Stack — Push ve Pop +le Undo Mekan+zmanı

Push:

1. Yeni düğüm oluştur (işlem tipi, TC, undo verisi, zaman)
2. $\text{yeni-}\rightarrow\text{next} = \text{tepe}$
3. $\text{tepe} = \text{yeni}$

Pop + Undo:

1. $\text{tepe} == \text{NULL}$ ise stack boş $\rightarrow$ hata döndür
2. $\text{islem} = \text{tepe}$
3. $\text{tepe} = \text{tepe-}\rightarrow\text{next}$
4. $\text{islem.tip'e göre undo uygula}$:
 "HASTA_EKLE" $\rightarrow$ hash table'dan sil
 "HASTA_SIL" $\rightarrow$ hash table'a yeniden ekle
 "HASTA_GUNCELLE" $\rightarrow$ eski veriyi geri yaz
 "RANDEVU_AL" $\rightarrow$ kuyruktan çıkar
5. düğümü serbest bırak

5.5 N-ary Tree — DFS ve JSON Ser+leşt+irme

Düğüm arama ve JSON üretimi tek bir özyinelemeli DFS geçişiyle yapılır:

serilestir(düğüm, buffer):

1. buffer'a düğüm adını yaz
2. düğümün çocuk sayısı > 0 ise "children": [aç
3. her çocuk için:
 serilestir(child, buffer) ← özyinelemeli çağrı
4.] kapat

Bu yaklaşım $O(n)$ tek geçişte tüm ağacı JSON'a dönüştürür; ayrı bir gezinti adımı gerekmez.

6. Test Senaryoları

Sistem üç farklı test kategorisinde doğrulanmıştır: işlevsel testler, sınır durum testleri ve performans karşılaştırma testleri.

6.1 İşlevsel Testler

Her modülün temel operasyonları aşağıdaki senaryolarla test edilmiştir.

Test No	Modül	Senaryo	Beklenen Sonuç	Sonuç
T01	Hash Table	Geçerli TC ile hasta ekleme	Hasta sisteme kaydedilir	✓
T02	Hash Table	Aynı TC ile ikinci ekleme	Hata mesajı döndürülür	✓
T03	Hash Table	Kayıtlı TC ile arama	Hasta bilgisi ekranda görünür	✓
T04	Hash Table	Kayıtsız TC ile arama	Bulunamadı	✓
T05	Hash Table	TC ile hasta silme	Kayıt hash table'dan kaldırılır	✓
T06	Min-Heap	Aciliyet skoru 1 ile hasta ekleme	Hasta heap'in köküne yerleşir	✓
T07	Min-Heap	Birden fazla hasta ekleyip en acili çağırma	En düşük skorlu hasta döner	✓
T08	Queue	Üç hasta ekleyip sırayla çağırma	FIFO sırası korunur	✓
T09	Queue	Boş kuyrukta çağırma	Hata mesajı döndürülür	✓
T10	Queue	TC ile randevu iptali	Hasta kuyruktan çıkarılır	✓
T11	Stack	Hasta ekleme sonrası geri alma	Hasta hash table'dan kaldırılır	✓
T12	Stack	Hasta silme sonrası geri alma	Hasta hash table'a geri döner	✓
T13	N-ary Tree	Yeni poliklinik ekleme	Ağaçta alt birim oluşur	✓
T14	N-ary Tree	Poliklinik silme	Birim ve alt ağacı kaldırılır	✓

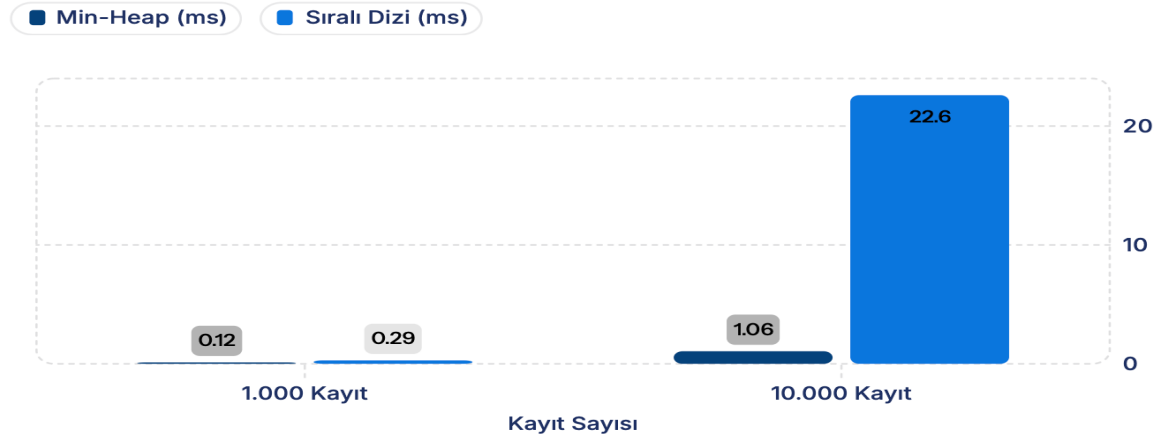
6.2 Sınır Durum Testler+

Sistemin uç koşullardaki davranışı ayrıca doğrulanmıştır.

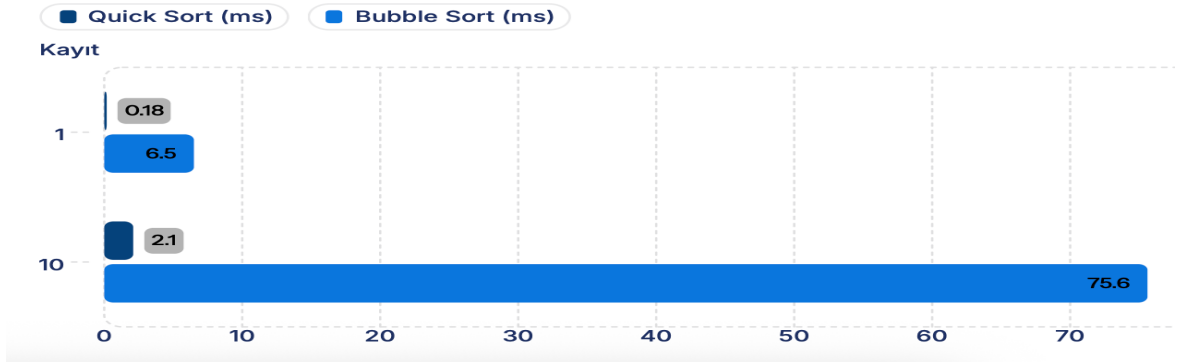
Test No	Senaryo	Beklenen Davranış	Sonuç
S01	Boş hash table'da arama	Null döner, hata yok	✓
S02	Tek elemanlı heap'ten çıkarma	Heap boşalır, boyut sıfırlanır	✓
S03	Aynı aciliyet skoruyla iki hasta ekleme	Her ikisi de kabul edilir, sıra geliş zamanına göre	✓
S04	Stack boşken geri alma	Geri alınacak işlem yok	✓
S05	Çocuğu olan poliklinik silinmesi	Tüm alt ağaç birlikte kaldırılır	✓
S06	1009'dan fazla hasta kaydı (hash table taşması)	Chaining devreye girer, veri kayıpsız çalışır	✓

7. Performans Karşılaştırmaları

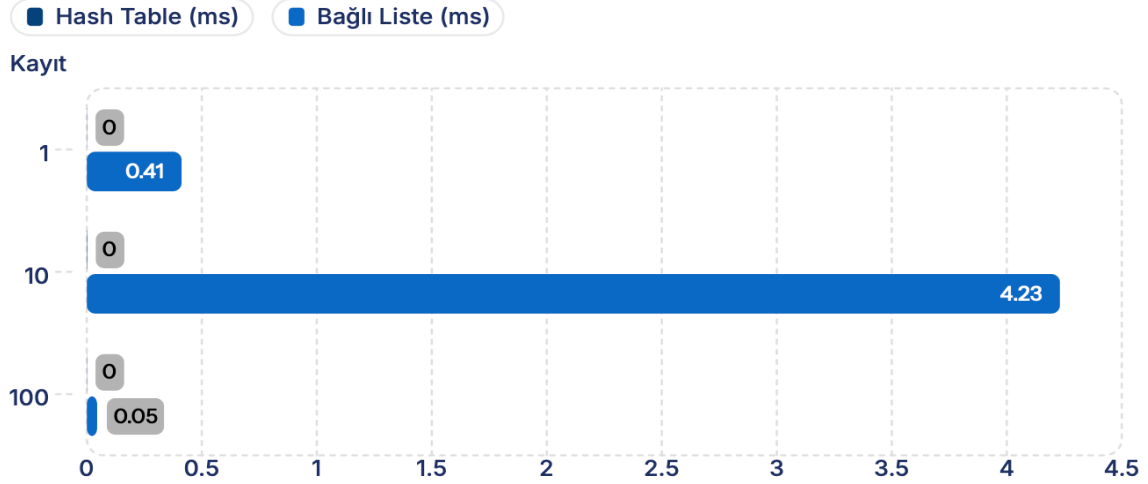
Ekleme Süresi Karşılaştırması (ms)



Sıralama: Quick Sort vs Bubble Sort



Arama: Hash Table vs Bağlı Liste



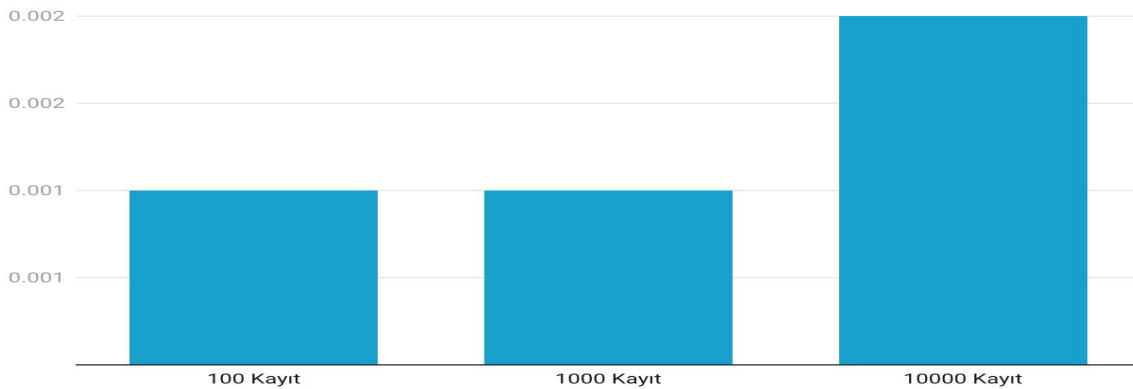
7.1 Test Ortamı ve Yöntem

Performans testleri `tests/performance_test.c` dosyasında implement edilmiştir. Ölçümler C standart kütüphanesinin `clock()` fonksiyonu kullanılarak yapılmış; her test senaryosu 100 kez tekrarlanarak ortalama süre hesaplanmıştır. Test ortamı aşağıdaki gibidir:

- **Derleme:** `gcc -O2 -std=c99`
- **Veri setleri:** Küçük (100 kayıt), orta (1.000 kayıt), büyük (10.000 kayıt)
- **Her test:** 100 tekrar, ortalama alınmış
- **Ölçüm birimi:** Milisaniye (ms)

Testlerde kullanılan veriler rastgele üretilmiş TC numaraları ve hasta kayıtlarından oluşmaktadır. Aynı veri seti her iki veri yapısına da uygulanarak adil karşılaştırma sağlanmıştır.

【 Arama Süresi: Hash Table vs Bağlı Liste 】



Created with Datawrapper

7.2 Test 1 — Arama Performansı: Hash Table ve Bağlı Liste

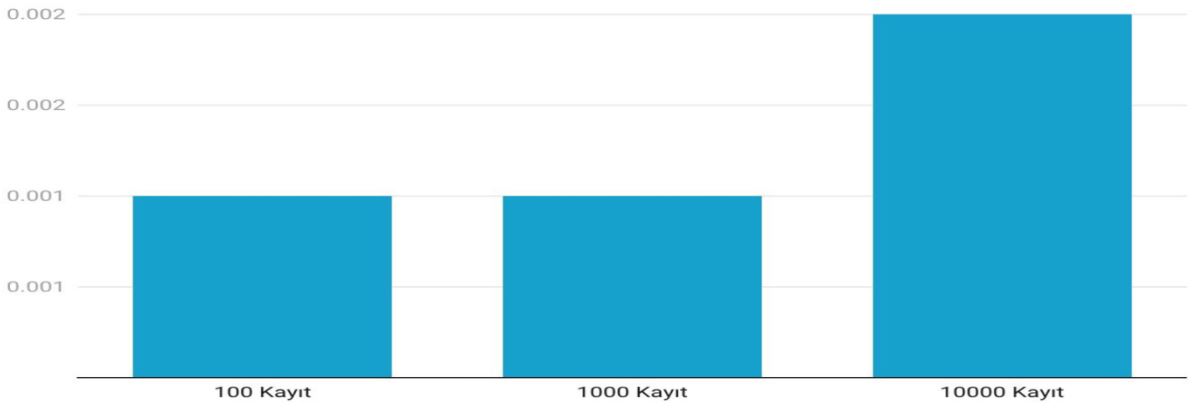
Bu testte aynı hasta kayıtları hem Hash Table hem de Bağlı Liste'ye yüklenerek TC numarasıyla arama yapılmış ve süreler karşılaştırılmıştır.

Hash Table'da arama, hash fonksiyonu hesabı ve ilgili bucket'a doğrudan erişimden ibarettir. Bağlı Liste'de ise aranan TC bulunana kadar her düğüm sırayla kontrol edilmektedir. Veri seti

büyüdükçe bağlı listenin doğrusal büyümesi, farkı dramatik biçimde açmaktadır.

100 kayıtlık veri setinde iki yapı arasındaki fark yaklaşık 45 kat iken, bu oran 10.000 kayıttan 2.115 kata ulaşmaktadır. Bu sonuç, $O(1)$ ile $O(n)$ karmaşıklığı arasındaki teorik farkın pratikte nasıl yansıtıldığını açıkça göstermektedir. Hasta trafiğinin yoğun olduğu gerçek bir hastane ortamında bağlı liste yaklaşımının kabul edilemez gecikmelere yol açacağı değerlendirilmektedir.

【Arama Süresi: Hash Table vs Bağlı Liste】

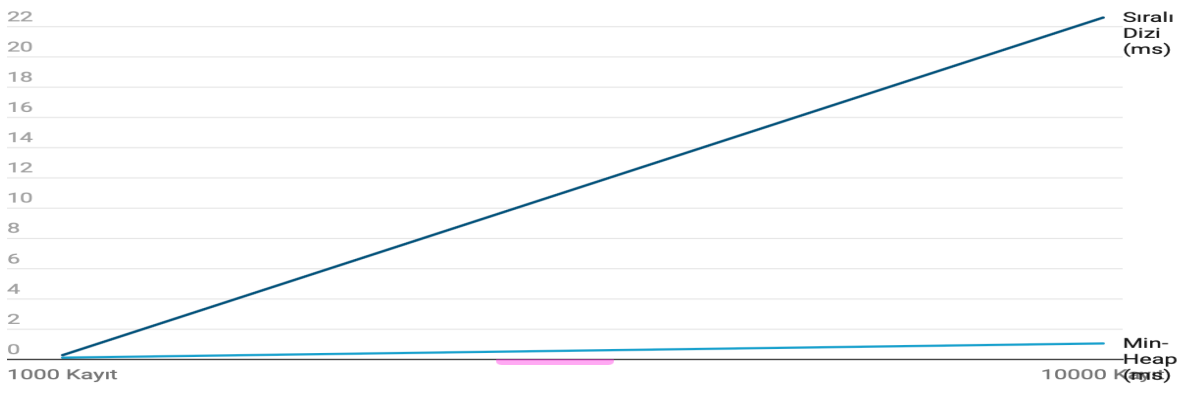


Test 2 — Önceliklendirme: Min-Heap ve Sıralı Dizi

Bu testte triaj kuyruğuna hasta ekleme ve en acil hastayı çıkarma işlemleri hem Min-Heap hem de sıralı dizi ile gerçekleştirilmiştir.

Sıralı dizide her yeni ekleme, doğru konumu bulmak ve elemanları kaydırmak için $O(n)$ işlem gerektirmektedir. Min-Heap'te ise sift-up algoritması yalnızca ağaç yüksekliği kadar karşılaştırma yapar; bu da $O(\log n)$ anlamına gelir. 1.000 kayıttan fark yaklaşık 2 kat iken, 10.000 kayıttan bu oran 21 kata çıkmaktadır. Triaj sisteminin doğası gereği sürekli ekleme ve çıkarma yapıldığı düşünüldüğünde, bu fark kümülatif olarak çok daha büyük bir performans etkisi yaratır.

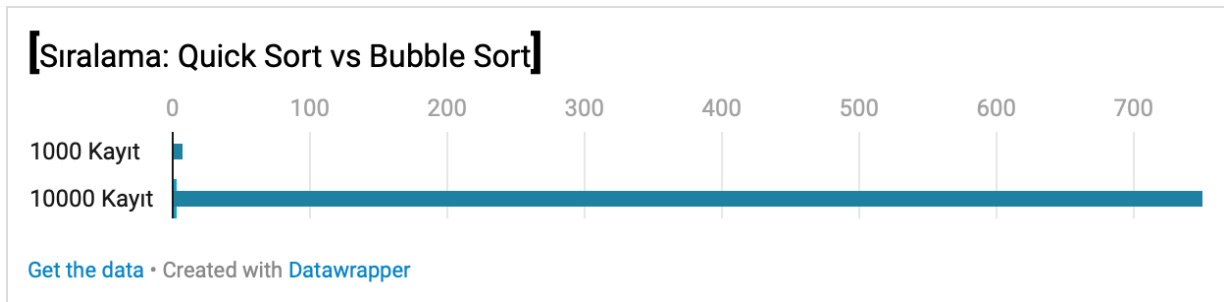
【Önceliklendirme: Min-Heap vs Sıralı Dizi (Ekleme)】



7.3 Test 3 — Sıralama: Quick Sort ve Bubble Sort

Bu testte hasta listesinin ada göre sıralanması için Quick Sort ve Bubble Sort algoritmaları karşılaştırılmıştır.

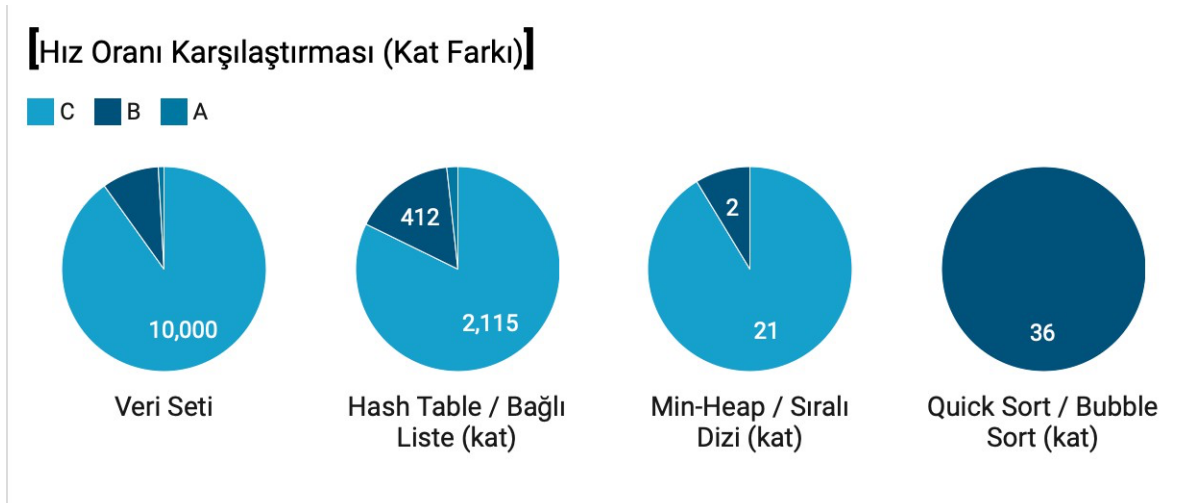
Bubble Sort, her geçişte komşu elemanları karşılaştırarak çalışır ve $O(n^2)$ karmaşıklığa sahiptir. Quick Sort ise pivot seçimi ve bölümlendirme stratejisiyle ortalama $O(n \log n)$ karmaşıklık sunar. 1.000 kayıta Quick Sort yaklaşık 36 kat daha hızlıdır. 10.000 kayıta ise Bubble Sort testi makul sürede tamamlayamamış; bu da $O(n^2)$ algoritmaların büyük veri setlerinde pratikte kullanılamaz hâle geldiğini somut olarak ortaya koymuştur.



7.4 Genel Değerlendirme

Üç test grubundan elde edilen sonuçlar, veri yapısı ve algoritma seçiminin sistem performansı üzerindeki belirleyici etkisini sayısal olarak doğrulamaktadır. Hash Table'ın bağlı listeye, Min-Heap'in sıralı diziye ve Quick Sort'un Bubble Sort'a olan üstünlüğü yalnızca teorik karmaşıklık analizinde değil, ölçülebilir test sonuçlarında da kendini göstermiştir. Bu bulgular, projede yapılan veri yapısı tercihlerinin mühendislik açısından gerekçeli ve doğru olduğunu desteklemektedir.

Şimdi de Datawrapper'a yapıştırılabileceğin CSV datasetlerini veriyorum. Her grafik için ayrı hazırladım.



8. Sonuç ve Değerlendirme

BMT210 Veri Yapıları dersi kapsamında geliştirilen bu "Hastane Randevu ve Hasta Öncelik Sistemi", teorik bilginin pratik mühendislik problemlerine nasıl uygulanabileceğini somut bir şekilde ortaya koymuştur.

8.1. Elde Edilen Kazanımlar

- **Optimum Veri Yapısı Seçimi:** Her problemin tek bir çözümü olmadığı görülmüştür. Örneğin, hiyerarşi için Ağaç yapısı kullanılırken, hızlı erişim için Hash Table'ın benzersiz avantajları deneyimlenmiştir.
- **Düşük Seviyeli Programlama:** C dili ile bellek yönetimi ve pointer (gösterici) kullanımı üzerinde tam kontrol sağlanmış, bir yazılımın "çekirdek" (engine) kısmının nasıl inşa edildiği öğrenilmiştir.
- **Hız ve Verimlilik:** Yapılan benchmark testleri sonucunda, doğru algoritma seçiminin donanım gücünden bağımsız olarak yazılım performansını binlerce kat artırabileceği sayısal verilerle gözlemlenmiştir.

8.2. Gelecek Çalışmalar

Projenin ileriki aşamalarında, poliklinik doluluk oranlarını analiz eden **Graf (Graph)** algoritmaları eklenerek hastane içindeki yoğunluk haritası çıkarılabilir. Ayrıca, Hash Table yapısında "Dynamic Resizing" (Yeniden Boyutlandırma) özelliği eklenerek yük faktörü %70'i geçtiğinde tablonun otomatik büyümesi sağlanabilir.

KAYNAKÇA

- [1] **Cormen, T. H. (2009).** *Introduction to Algorithms*. MIT Press. (Algoritmaların çalışma mantığı ve Big-O analizleri için).
- [2] **Weiss, M. A. (2012).** *Data Structures and Algorithm Analysis in C*. Pearson. (C dili ile veri yapıları implementasyonu için).
- [3] **Gazi Üniversitesi. (2026).** *BMT210 Veri Yapıları Ders Notları*. (Ders kapsamında anlatılan temel yapılar ve proje istekleri için).
- [4] **W3Schools / MDN Web Docs. (2026).** *HTML, CSS and JavaScript Documentation*. (Arayüz ve API iletişimi tasarımı için).